ITEC618 – Programming Concepts

Assessment 3: Programming assignment

Semester 1, 2025

Submitted by:

Amir Tamang

Student ID: S00393664

Email: S00393664@myacu.edu.au

Date of Submission: 22/05/2025

Actual Word Count: 2277

Part 1: Object-Oriented Programming (OOP) Concepts and Their Real-World Applications

OOP is one of the most powerful programming paradigms, where everything exists as an object that can contain data, in the form of fields (also referred to as attributes), and code, in the form of procedures (also called methods). It's mainly used to make your app more reusable/modular/scalable. During weeks 8 to 11, I learned basic OOP concepts in the ITEC618 Programming Concepts unit. Four of these concepts and their applications to the real world and the common good are examined in this report:

1. Encapsulation
2. Inheritance
3. Polymorphism
4. Abstraction

1. Encapsulation

Encapsulation is a mechanism of wrapping the data (variables) and code acting on the data (methods) together as a single unit and hiding the object's state from the outside world. This is usually accomplished through access modifiers such as private, protected, and public.

For example:

```
public class BankAccount {
    private double balance;
    public BankAccount(double initialBalance) {
        this.balance = initialBalance;
    }
    public void deposit(double amount) {
```

```
        if (amount > 0) balance += amount;
    }
    public double getBalance() {
        return balance;
    }
}
```

In this case, the variable balance is private, which means you cannot alter the balance from outside the class. This hides the internal state and requires access through the methods.

2. Inheritance

Inheritance is a way to form new classes (instances) using classes that have already been defined. This also allows to share code and build a natural hierarchy.

Example:

```
public class Product {
    protected String name;
    protected double price;
    public Product(String name, double price) {
        this.name = name;
        this.price = price;
    }
    public void displayInfo() {
        System.out.println(name + ": $" + price);
    }
```

```
}
public class FreshFruit extends Product {
    private double weight;
    public FreshFruit(String name, double price, double weight) {
        super(name, price);
        this.weight = weight;
    }
    public void displayFruitInfo() {
        displayInfo();
        System.out.println("Weight: " + weight + "g");
    }
}
```

Here's FreshFruit which inherits from Product, so that we can reuse the name, price, and displayInfo().

3. Polymorphism

Polymorphism enables a single interface to be used for an entire class of actions. It allows methods to act differently based on the object that calls them.

Example:

```
Product p1 = new FreshFruit("Apple", 1.5, 100);
Product p2 = new PackagedItem("Cookies", 3.0, 10, "2025-05-01");
System.out.println(p1.toString());
System.out.println(p2.toString());
```

Although p1 and p2 are of type Product, in their toString() they invoke toString methods of FreshFruit and PackagedItem. This is a runtime polymorphism by method overriding.

4. Abstraction

Abstraction is the methodology of presenting only the necessary features and hiding the rest of the complex implementation detail. In Java, you can do this via abstract class and interface.

Example:

```
public abstract class Product {
    protected String name;
    protected double price;
    public abstract void printDetails();
}
```

Any Product subclass must define the printDetails() method. This imposes a uniform interface but permits variety of implementation.

Real-World Applications and Common Good

OOP concepts allow for highly modularized applications across numerous fields. Financial institutions leverage OOP design in online banking platforms through classes that model users' accounts, transfers, and statements. Similarly, electronic health platforms organize patient data, appointments, and insurance details within class structures. E-commerce giants depend on object-oriented e-commerce systems, with classes for varying items, carts, payments, and user profiles. Within such a system, one might find complex product classes containing inventories, pricing, images and full descriptions; composite order classes tying customers to lists of goods

and total charges; and polymorphic payment classes for diverse transaction methods. Well-crafted object models like this deliver scalability and flexibility and simplify the integration of new features.

The impact of OOP systems from a social perspective relates to the capability to promote scalable and maintainable software for the common good. Well-designed OOP highly reduces bugs, downtime, increases accessibility, and allows for strategic longevity/development. For example, in our local public services such as transportation and health services, OOP-based systems can facilitate faster processing of data and make better decisions on OOP based systems. These impacts can lead to better public access/services and improved public safety.

References

Oracle, 2024. *Oracle.com.* [Online]
Available at: https://docs.oracle.com/javase/tutorial/java/concepts/
[Accessed 30 05 2025].

W3schools, 2025. *W3schools.* [Online]
Available at: https://www.w3schools.com/java/java_oop.asp
[Accessed 30 05 2025].

Y.D, L., 2018. *Introdution to Java Programming and Data Structures.* s.l.:Pearson Education.